短縮タグGCMに対する認証鍵回復攻撃の改良

井上 明子[1,a)]　岩田 哲[2]　峯松 一彦[1,3]

概要：本論文では NIST 標準認証暗号の GCM に対する新たな認証鍵回復攻撃を提案する．GCM は改ざん検知の安全性レベルが入力長に比例して下がることが知られており，特に短縮タグを用いる時にその低下は顕著となる．実際に Ferguson は τ ビットタグ GCM で 2^d ブロック入力を用いることで，復号クエリ $2^{\tau-d}$ 回で改ざん攻撃が成功することを示し，且つこの攻撃を繰り返すことでデータ計算量 $O(2^\tau)$ で GCM 内部の認証鍵を回復できることを示した．これを踏まえて NIST は 32/64 ビットタグ GCM の入力長と復号関数呼出回数を厳しく制限した．これにより認証鍵回復に必要なデータ計算量は，32/64 ビットタグでそれぞれ 2^{61}〜2^{67}，2^{70}〜2^{75} に増大すると従来研究では見積もられていた．本稿で提案する認証鍵回復攻撃は，これらの計算量を $O(2^{\tau+1})$ に削減できることを示す．提案攻撃に必要なデータ計算量は最良のケースで，NIST の何れの制限下でも 32/64 ビットタグの場合にそれぞれ 2^{33}，2^{65} となる．これは NIST の制限が認証鍵回復の計算量増加にほぼ寄与しないことを示している．また，提案攻撃はナンス誤用設定下においても適用可能であり，これは Joux の forbidden attack を初めて短縮タグの場合に拡張した攻撃である．

Refinements of Authentication Key Recovery Attack against GCM

Akiko Inoue[1,a)]　Tetsu Iwata[2]　Kazuhiko Minematsu[1,3]

Abstract: This paper presents an authentication key recovery attack on GCM. Ferguson's seminal forgery attack on GCM shows that the authenticity bound $\ell q/2^\tau$ of GCM is tight, where τ is an authentication tag length, q is the number of decryption queries, and ℓ is the maximum number of input blocks consisting of AD and ciphertext. Ferguson also showed that the authentication key can be fully recovered by repeating such forgeries; thus, the required query complexity is $O(2^{\tau-\log_2 \ell})$ and the data complexity is $O(2^\tau)$. Given that situation, NIST heavily restricted the length of inputs and the number of invocations of the decryption function for the 32/64-bit tag cases. Under these limits, Mattsson and Westerlund showed that the data complexity for key recovery increases to 2^{61}–2^{67} and 2^{70}–2^{75} for the 32-bit tag and 64-bit tag cases. Our authentication key recovery attack on GCM reduces these complexities to $O(2^{\tau+1})$. In the best-case scenario, our attack achieves data complexities of 2^{33} and 2^{65} for the 32-bit and 64-bit tag cases under any of the current NIST's limits. This indicates that the current NIST's limits do not substantially increase the data complexity for authentication key recovery. We also apply our proposed attack to the nonce-misuse setting, which extends Joux's forbidden attack to the truncated-tag case for the first time.

1. はじめに

認証暗号 [3, 21] は秘匿と改ざん検知を同時に実現する共通鍵暗号方式であり，TLS や WPA 等の実世界上のプロトコルで広く用いられている．NIST 標準認証暗号の GCM（NIST SP 800-38D [7]）は実世界で最も広く用いられている認証暗号の 1 つであり，TLS，IPsec，SSH 等世界中の多様な場面で用いられている．その広い適用範囲に反して GCM にはいくつかの弱点があることが指摘されており，特に改ざん検知機能を実現している認証部分に関して多くの解析がなされている．例えばナンス誤用による認証鍵回復 [12] や，復号/暗号化クエリによる認証鍵の絞り込み及び回復 [1, 9, 13, 14, 16, 19, 20, 22]，短縮タグの場合における安全性低下 [8, 15] などが挙げられる．本稿では短縮タグ GCM に対する解析に焦点を当てる．

GCM の安全性バウンド [11, 18] には $\ell q/2^\tau$ という項が

[1] NEC
[2] 名古屋大学, Nagoya University
[3] 大阪大学, The University of Osaka
[a)] a_inoue@nec.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

表 1: NIST SP 800-38D [7] で規定された入力長制限及び復号クエリ数制限下における，認証鍵回復に必要なデータ計算量の対数．ここで full block とは元の AD 及び暗号文の長さが何れも 128 の倍数である場合を意味し，partial block とは，そのうちの少なくとも一方が 128 の倍数でない場合を意味する．

τ	32						64					
$\log_2 \ell$	1	2	3	4	5	6	11	13	15	17	19	21
$\log_2 q$	22	20	18	15	13	11	32	29	26	23	20	17
Ferguson [8]	62	62	63	66	69	72	79	79	79	80	80	81
MW [15]	62	62	61	65	66	67	75	74	73	72	71	70
Ours (full block)	**33**	**33**	**33**	**33**	**33**	**33**	**65**	**65**	**65**	**65**	**65**	**65**
Ours (partial block)	42	43	35	35	35	35	67	67	67	67	67	67

表 2: ナンス誤用設定下における各認証鍵回復攻撃の比較．何れの攻撃も受動的攻撃者を仮定しており，攻撃者は AD と暗号文が合わせて高々 ℓ ブロックの，ナンス，AD，暗号文，タグのタプルを 2 つ取得する．また提案攻撃は full block，且つ 2 つの AD と暗号文がそれぞれ同じ長さであることを仮定する．$\mathsf{equsol}(\ell)$ は次数 ℓ の方程式を解くための計算量であり，$\mathsf{find}(\ell)$ は ℓ 個の鍵候補から本物の認証鍵を見つけるために必要なクエリ数である．復号クエリを用いる場合は $\mathsf{find}(\ell) \geq \log_2(\ell)$ であり，ナンス誤用暗号化クエリを用いる場合は平均的に $\mathsf{find}(\ell) \approx \log_2(\ell)/\tau$ である．

	適用タグ長	オフライン計算量	追加クエリ数
Original Joux [12]	128 bits only	$\mathsf{equsol}(\ell)$	$\mathsf{find}(\ell)$
Extended Joux (6.1 節)	any τ	$2^{128-\tau}\mathsf{equsol}(\ell)$	$\mathsf{find}(2^{128-\tau}\ell)$
Ours (6.2 節)	any τ	$\mathsf{equsol}(\ell)$	復号クエリ $128-\tau$ 回 + $\mathsf{find}(\ell)$

含まれる．ここで τ は認証タグの長さ，q は復号クエリ数，ℓ は AD と暗号文からなる入力ブロックの最大数である．この項は τ ビットタグを持つ認証暗号に対して予想される理想的なバウンド $q/2^\tau$ と比較すると，改ざん検知の安全性レベルを大きく低下させている．実際，Ferguson [8] は 2^d ブロックの AD/暗号文入力を用いると確率 $1/2^{\tau-d}$ で偽造攻撃が成功することを示し，バウンド $\ell q/2^\tau$ がタイトであることを示した．また同論文で Ferguson は，この偽造攻撃を成功させることで GCM 内部の認証鍵に関する線形方程式が漏洩することを示し，これを繰り返すことで認証鍵の完全な復元が可能になることを示した．更に，偽造攻撃の成功はその後の偽造攻撃の成功確率を高めることを示した．例えば $\tau = 32$, $d = 16$ の場合，最初の偽造には 2^{16} 回の復号クエリが必要であり，偽造成功によって認証鍵に関する 16 個の線形方程式が明らかになる[*1]．その後の偽造には 2^{14} 回，2^{12} 回，2^9 回，2^6 回，... の復号クエリが必要となる．つまり，Ferguson の認証鍵回復攻撃には $O(2^{\tau-d})$ 回の復号クエリが必要であり，必要なデータ計算量は $O(2^d \cdot 2^{\tau-d}) = O(2^\tau)$ となる．

この攻撃を緩和するため，NIST SP 800-38D [7] では $\tau = 32, 64$ の場合について，最大入力長及び鍵ごとの GCM 復号関数の最大呼出回数を厳しく制限している．Mattsson と Westerlund [15] は，これらの制限下における Ferguson の認証鍵回復攻撃の必要データ計算量を解析した．その結果は表 1 の通り，32 ビットタグの場合は 2^{62}〜2^{72}，64 ビットタグの場合は 2^{79}〜2^{81} となっており，制限のない場合のデータ計算量 2^τ と比較して大幅に増加していることが明らかとなった．この増分の原因は，Ferguson 攻撃は 1 つの認証鍵を完全回復するために，最初の偽造成功後も高コストな偽造攻撃（*i.e.*, NIST によるクエリ回数制限を大きく上回るクエリ数が必要な偽造攻撃）の成功を複数回必要することにある．また，Mattsson と Westerlund [15]（MW）は最初の偽造攻撃に必要なクエリ回数と 2 回目以降の偽造攻撃に必要なクエリ回数の間でトレードオフを取ることで，NIST 制限下における認証鍵回復攻撃に必要なデータ計算量を改善している．しかし，2 回目以降の偽造攻撃は依然として高コストであり，その結果，表 1 に示す通り，MW の認証鍵回復攻撃に必要なデータ計算量は 2^τ を大幅に上回ったままである．

最近 NIST は NIST SP 800-38D の改訂を進めており，改訂版では 96 ビット未満のタグへの対応を削除すると発表した [17] が，短いタグを用いた GCM は依然として実用上重要な位置を占めている．特に，IPsec ESP [23] 及び IKE v2 [4] では，64 ビットのタグを用いた GCM の使用が許可されている．一般的に短縮タグは，帯域幅やリソースに制約のある環境（例えば Zigbee では 32 ビットのタグが使用されているが，GCM [7] の代わりに CCM が使用されている）や，リアルタイム通信プロトコルにおいて魅力的である．更に NIST は Rijndael-256 [6] の標準化に伴い，GCM

[*1] 元の入力ビット長が 128 の倍数でない場合．そうでない場合は，2^{17} 回の復号クエリが必要となり，偽造成功で 17 個の線形方程式が漏洩する．

© 2026 Information Processing Society of Japan

の内部で用いるブロック暗号の入出力長を 128 ビットから 256 ビットに広げた方式である wGCM を規定することも発表している [17]．wGCM の仕様はまだ厳密には定まってはいないが，NIST は短縮タグの利用について言及している [17]．これより，wGCM が GCM の設計仕様を引き継ぎ，且つ 256 ビット未満の短縮タグを用いる可能性は大いにあると考えられる．このような側面から，短縮タグが GCM の安全性に及ぼす影響をより深く解析することは，NIST の改訂計画における実用的な意義や安全性を評価する上でも大いに役立つと考えられる．

1.1 本稿の貢献

本稿では GCM に対する新たな認証鍵回復攻撃を提案し，NIST 制限下における認証鍵回復に必要なデータ計算量を既存攻撃から大幅に削減する．表 1 に示すように，最良の場合，提案攻撃は NIST の何れの制限下でも任意の τ に対して $O(2^{\tau+1})$ のデータ計算量を達成する．つまり NIST の制限は認証鍵回復のデータ計算量増加にほぼ寄与していないことが分かる．

提案攻撃は，Ferguson/MW 攻撃において 2 回目以降の高コスト偽造攻撃に必要なクエリ数を単一の復号クエリに削減する．Ferguson/MW 攻撃では各偽造毎に全く新しい AD/暗号文を再生成するのに対し，提案攻撃では 2 回目以降の偽造で用いる AD/暗号文を最初の偽造の AD/暗号文から導出する．これにより，最初の偽造成功から得られた情報を最大限活用することが可能となり，正当なタグの未知ビット数が 1 となるような AD/暗号文を構築できる．これを繰り返すことで，攻撃者は最初の偽造を成功させた後，1 回の復号クエリ毎に認証鍵に関する線形方程式を 1 つ収集することができる．この線形方程式を 1 つずつ蓄積する手法は，Inoue ら [10] が提案した GCM-SST [5] に対する汎用偽造攻撃におけるビット毎の認証サブ鍵回復の一般化と見なすことができる．

また，提案攻撃はナンス誤用設定下においても適用可能である．GCM はナンスを誤用，つまり暗号化関数に同じナンスを繰り返し入力してしまうと認証鍵が漏洩する [12]．これは Joux の forbidden attack として広く知られているが，Joux はタグを短縮しない場合の攻撃のみを示しており，短縮タグの場合の攻撃は我々の知る限り既存文献では議論されていない．Joux の攻撃はタグが大きく短縮されると効果を著しく損なうが，そのようなケースにおける認証鍵回復の計算量評価は未知であった．本稿では提案攻撃をナンス誤用設定下に適用することで，Joux の forbidden attack を初めて短縮タグの場合に拡張する．

2. 準備

ビット列 X について $|X|$ をそのビット数とし，非負整数 $s \leq |X|$ に対して $\mathtt{msb}_s(X)$ (*resp.* $\mathtt{lsb}_s(X)$) を X の上位 (*resp.* 下位) s ビットとする．$X \ll s$ (*resp.* $X \gg s$) を X の s ビット左シフト (*resp.* 右シフト) とする．ビット列 X と X' の連結を $X \,||\, X'$ で表し，全ビットが 0 である s ビットの文字列を 0^s と表記する．

2.1 認証暗号 [3, 21]

認証暗号 Π は暗号化関数 $\Pi.\mathcal{E}$ と復号関数 $\Pi.\mathcal{D}$ により定義される．鍵空間を $\mathcal{K}$，ナンス空間を $\mathcal{N} = \{0,1\}^\nu$，Associated Data (AD) 空間を $\mathcal{AD}$，平文/暗号文空間を $\mathcal{M}$，タグ空間を $\mathcal{T} = \{0,1\}^\tau$ とすると，暗号化関数は $\Pi.\mathcal{E}\colon \mathcal{K}\times\mathcal{N}\times\mathcal{AD}\times\mathcal{M} \to \mathcal{M}\times\mathcal{T}$ として定義され，復号関数は $\Pi.\mathcal{D} : \mathcal{K}\times\mathcal{N}\times\mathcal{AD}\times\mathcal{M}\times\mathcal{T} \to \mathcal{M}\cup\{\perp\}$ として定義される．ここで，AD は暗号化の必要はないが改ざんは検知したいデータであり，例えばヘッダー情報である．また，$\perp$ は入力に改ざんが検知されたことを示す単一のエラーメッセージである．また，$\Pi.\mathcal{E}_K(N,A,M) = (C,T)$ のとき $\Pi.\mathcal{D}_K(N,A,C,T) = M$ となることが必要である．

2.2 GCM [7]

GCM のアルゴリズムを図 1, A·1 に示す．GCM を定義する有限体は $\mathrm{GF}(2^{128}) = \mathrm{GF}(2)[x]/(f(x))$ であり，ここで $f(x) = x^{128}+x^7+x^2+x+1$ である．128 ビット値 A について，$a_0a_1\cdots a_{127}$ を A を構成するビット列とする．GCM は 128 ビット値をリトルエンディアン形式で多項式に変換する．つまり $A = a_0 + a_1x + \cdots + a_{127}x^{127}$ となる．A に 2^{-1} (*i.e.*, x^{-1}) を掛けることは，$\mathtt{msb}_1(A) = a_0 = 0$ の場合には 1 ビット左シフト $A \ll 1$ に対応し，それ以外の場合は $2^{-1}A = (A \ll 1) \oplus 1^20^410^{120}1$ となる．

3. 提案攻撃

3.1 概要

提案攻撃の概要を以下に示す．提案攻撃は 3 つのステップで構成される．まず第 1 ステップとして攻撃者は，Ferguson の偽造攻撃 [8] によって 2 つの正当なタプル (N,A,C,T) 及び (N,A_1,C_1,T_1) を取得する．ここで N はナンス，A と A_1 は AD，C と C_1 は暗号文，T と T_1 はタグであり，**正当な**タプルとは，このタプルを復号関数に入力すると $\perp$ 以外が出力されるものである．但し $|A|, |C|, |A_1|, |C_1|$ は全て 128 の倍数であり，$|A| = |A_1|$, $|C| = |C_1|$，且つ $|A|/128 + |C|/128 + 1 = 2^d$ (d は正の整数) とする．Ferguson の攻撃では GHASH 出力差分 $\bigoplus_{i=1}^{x}(X[i] \oplus X_1[i])L^i$ を利用する．この多項式は**エラー多項式**と呼ばれ，短縮前のタグの差分に相当する．ここで L は回復対象の認証鍵であり，$X = A \,||\, C \,||\, \langle|A|\rangle_{64} \,||\, \langle|C|\rangle_{64} = X[x] \,||\, \cdots \,||\, X[1]$ として定義され，X_1 も同様に定義される．そして i が 2 の冪でない場合に $X[i] \oplus X_1[i] = 0^{128}$ と設定することで，エラー多項式を線形化することができる．これは，GCM が標数 2 の体上で定義されており，定数乗算及び二乗が線

© 2026 Information Processing Society of Japan

Algorithm $\mathsf{GCM}.\mathcal{E}[E_K](N, A, M)$
(1) $C \leftarrow \mathsf{GCTR}[E_K](N, M)$
(2) $T \leftarrow \mathsf{GMAC}[E_K](N, A, C)$
(3) **return** (C, T)
Algorithm $\mathsf{GCTR}[E_K](N, M)$
(1) $L \leftarrow E_K(0^n)$
(2) **if** $\nu = 96$ **then** $\mathsf{Ctr0} \leftarrow N \,||\, 0^{31}1$
(3) **else then** $\mathsf{Ctr0} \leftarrow \mathsf{GHASH}_L(\varepsilon, N)$
(4) $(M[1], \ldots, M[m]) \xleftarrow{n} M$
(5) **for** $i = 1$ **to** m **do**
(6) $\quad C[i] \leftarrow M[i] \oplus \mathtt{msb}_{|M[i]|}(E_K(\mathsf{inc}^i(\mathsf{Ctr0})))$
(7) $C \leftarrow C[1] \,||\, \cdots \,||\, C[m-1] \,||\, C[m]$
(8) **return** C
Algorithm $\mathsf{GMAC}[E_K](N, A, C)$
(1) $L \leftarrow E_K(0^n)$
(2) **if** $\nu = 96$ **then** $\mathsf{Ctr0} \leftarrow N \,||\, 0^{31}1$
(3) **else then** $\mathsf{Ctr0} \leftarrow \mathsf{GHASH}_L(\varepsilon, N)$
(4) $\mathsf{Mask} \leftarrow \mathtt{msb}_\tau(E_K(\mathsf{Ctr0}))$
(5) $T \leftarrow \mathsf{Mask} \oplus \mathtt{msb}_\tau(\mathsf{GHASH}_L(A, C))$
(6) **return** T
Algorithm $\mathsf{GHASH}_L(A, C)$
(1) $X \leftarrow \mathsf{pad}_n(A) \,||\, \mathsf{pad}_n(C) \,||\, \langle |A| \rangle_{n/2} \,||\, \langle |C| \rangle_{n/2}$
(2) $(X[x], \ldots, X[1]) \xleftarrow{n} X,\ Y \leftarrow 0^n$
(3) **for** $i = x$ **to** 1 **do** $Y \leftarrow L \cdot (Y \oplus X[i])$
(4) **return** Y

図 1: ブロック暗号 E_K を用いた時の GCM のアルゴリズム．復号関数は省略する．ここで $\nu := |N|, \tau := |T|$ は予め固定された値である．また，$n = 128$ であり，GHASH の 3 行目の "$\cdot$" は $\mathrm{GF}(2^n)$ 上の乗算である．

形演算であるためである．線形化されたエラー多項式を，$\mathrm{GF}(2)$ 上の 128×128 行列 A_D を用いて $A_D L$ と表記し，この A_D を**エラー行列**と呼ぶ．この第 1 ステップの目的は，$A_D L$ の上位 τ ビットに対応する $T \oplus T_1$ を求めることである．Ferguson の攻撃では，(A_1, C_1) を操作して A_D の上 $d-1$ 行を 0 にすることにより，T_1 の実質的なタグ長**（実効タグ長）**を $\tau - d + 1$ ビットに短縮する．

第 2 ステップでは Ferguson 攻撃や MW 攻撃とは異なり，第 1 ステップで使用した $A_D L$ から $i = 2, 3, \ldots$ に対する新しい偽造 (N, A_i, C_i, T_i) を導出する．第 1 ステップより $\mathtt{msb}_\tau(A_D L)$ は攻撃者にとって既知であることから，(A_2, C_2) に対応するエラー行列が $2^{-1} A_D$ となるように (A_2, C_2) を上手く操作することで，正当なタグ T_2 を $T \oplus \mathtt{msb}_\tau(2^{-1} A_D L)$ とできる．おおよそ $\mathtt{msb}_\tau(2^{-1} A_D L)$ は $\mathtt{msb}_\tau(A_D L \ll 1)$ であり，このうち未知ビット数は 1 ビットのみであるため，1 回の復号クエリで T_2 の正しい値を求めることができる．第 2 ステップではこの手順を繰り返して $A_D L$ の値全体を復元し，これにより第 1 ステップと併せて認証鍵に関する $128 - d + 1$ 個の線形方程式が明らかとなる．この $A_D L$ のビット毎の復元は，GCM-SST における認証サブ鍵の復元手法 [10] に着想を得たものである．

第 3 ステップでは Ferguson/MW 攻撃の 2 回目以降の偽造攻撃を用いて，残りの未知 $d-1$ ビットを復元する．1 章にて我々は Ferguson/MW 攻撃における 2 回目以降の偽造コストが高いことを指摘したが，提案攻撃では第 2 ステップまでに認証鍵に関する十分な線形方程式を取得済みである．したがって，T^* の実効タグ長が 1 となるような (N, A^*, C^*, T^*) を構成することができる．これを繰り返すことで，1 回の復号クエリで認証鍵に関する線形方程式を 1 つずつ得ることができ，残りの未知 $d-1$ ビットを復元できる．

3.2 攻撃詳細

第 1 ステップ: 最初の偽造． 攻撃者は暗号化クエリ (N, A, M) を実行し (C, T) を取得する．A 及び M/C のビット長は 128 の倍数であると仮定する．よって以下が成立する．

$$T = \mathtt{msb}_\tau(E_K(\mathsf{Ctr0}) \oplus \mathsf{GHASH}_L(A, C)) = \mathtt{msb}_\tau\left(E_K(\mathsf{Ctr0}) \oplus \bigoplus_{i=1}^{x} X[i] \cdot L^i\right).$$

ここで $X = A \,||\, C \,||\, \langle |A| \rangle_{64} \,||\, \langle |C| \rangle_{64} = X[x] \,||\, \cdots \,||\, X[1]$ であり $\mathsf{Ctr0}$ は N から導出される 128-bit 値である（詳細定義は図 1 を参照）．ここで正数 d に対して $x = 2^d$ を仮定する．このステップにおける攻撃者の目的は，Ferguson の偽造攻撃 [8] によって正当な偽造 (N, A_1, C_1, T_1) を得ることである．ここで $|A_1| = |A|$, $|C_1| = |C|$ とし，$X_1 = A_1 \,||\, C_1 \,||\, \langle |A_1| \rangle_{64} \,||\, \langle |C_1| \rangle_{64} = X_1[x] \,||\, \cdots \,||\, X_1[1]$ とすると，正当なタグ T と T_1 について以下が成立する．

$$T \oplus T_1 = \mathtt{msb}_\tau\left(\bigoplus_{i=1}^{x} X[i] L^i \oplus \bigoplus_{i=1}^{x} X_1[i] L^i\right) := \mathtt{msb}_\tau\left(\bigoplus_{i=1}^{x} \Delta_1[i] L^i\right).$$

ここで i が 2 の冪でない場合に $\Delta_1[i] = 0^n$ と設定すると上式は以下の通りとなる．

$$T \oplus T_1 = \mathtt{msb}_\tau\left(\bigoplus_{i=1}^{d} D[i] L^{2^i}\right).$$

ここで $D[i] = \Delta_1[2^i]$ である．但し，我々の攻撃では $|A| = |A_1|$, $|C| = |C_1|$ を設定しているため $D[0] = 0^n$ となることに注意されたい．Ferguson [8] が示した通り，定数倍と二冪計算は $\mathrm{GF}(2^{128})$ において線形であるため，上式は線形化できる．ここで $M_{D[i]}, M_S$ をそれぞれ，$D[i]$ による定数倍と二冪計算によって定義される $\mathrm{GF}(2)$ 上の 128×128 行列と定義すると，上式は

$$T \oplus T_1 = \mathtt{msb}_\tau\left(\bigoplus_{i=1}^{d} M_{D[i]} (M_S)^i L\right) := \mathtt{msb}_\tau\left(A_D L\right)$$

© 2026 Information Processing Society of Japan

となる．A_D は GF(2) 上の 128×128 行列である．攻撃者は $D[1], \ldots, D[d]$ を $128d$ 個のバイナリ変数として操作できるので，これらの変数に関する線形方程式を解くことで，攻撃者は A_D の上 $d-1$ 行が全て 0 になるように $D[i]$ を選択することができる．これにより $T \oplus T_1$ の上位 $d-1$ ビットは 0 となり，偽造のための実効タグ長は $\tau - d + 1$ ビットとなる．攻撃者は A_D を固定し，T_1 を変えて復号クエリを行うことで正当な偽造 (N, A_1, C_1, T_1) を得る．そして認証鍵 L に関する $\tau - d + 1$ 個の非自明な線形方程式を得る．このステップでは 1 回の暗号化クエリと $2^{\tau-d+1}$ 回の復号クエリが必要であり，各クエリで 2^d ブロックの入力が必要である．

第 2 ステップ: 線形方程式の収集. 第 1 ステップで攻撃者は $\mathtt{msb}_\tau(A_D L)$ を取得した．このステップでは $A_D L$ の値全体をビット毎に復元し，L に関する新たな $128-\tau$ 個の線形方程式を得る．攻撃者は $A_D L$ の $\tau+1$ 番目のビットを復元するために，$|A_2| = |A|, |C_2| = |C|$ なる新たな偽造 (N, A_2, C_2, T_2) を構成する．ここで $X_2 = A_2 \,||\, C_2 \,||\, \langle |A_2| \rangle_{64} \,||\, \langle |C_2| \rangle_{64} = X_2[x] \,||\, \cdots \,||\, X_2[1]$ とする．攻撃者は $i = 1, \ldots, d$ に対して $X_2[2^i] = X[2^i] \oplus 2^{-1} D[i]$ と設定し，二冪でない j に対して $X_2[j] = X[j]$ とする．このとき正当なタグ T_2 は次の等式を満たす．

$$T \oplus T_2 = \mathtt{msb}_\tau \left(\bigoplus_{i=1}^{d} 2^{-1} D[i] L^{2^i} \right) = \mathtt{msb}_\tau(2^{-1} \cdot A_D L).$$

攻撃者にとって $A_D L$ の上位 τ ビットは $T \oplus T_1$ として既知であり，GCM において 2^{-1} 乗算はほぼ 1 ビット左シフトに対応する．つまり b を $A_D L$ の $\tau+1$ 番目のビットとすると，$\mathtt{msb}_1(A_D L) = 0$ のとき

$$T \oplus T_2 = (\mathtt{lsb}_{\tau-1}(T \oplus T_1) \,||\, b)$$

が成立し，$\mathtt{msb}_1(A_D L) = 1$ のときは

$$T \oplus T_2 = (\mathtt{lsb}_{\tau-1}(T \oplus T_1) \,||\, b) \oplus \mathtt{msb}_\tau(1^2 0^4 10^{120} 1)$$

が成立する．よって攻撃者は 1 回の復号クエリから b を復元できる．復号クエリが失敗した場合，攻撃者はその反対のビットが正しい値であることが分かるためである．

上記手順を繰り返すことで，攻撃者は $A_D L$ の未知ビットを 1 ビットずつ復元し，L に関する $128 - \tau$ 個の線形方程式を新たに取得できる．以下詳細手順を述べる．いま，$A_D L$ の上位 $\tau + s$ ビットが復元されたと仮定する．つまり攻撃者は正当な偽造 $(N, A_{s+1}, C_{s+1}, T_{s+1})$ を構成済みであるとする．ここで $|A_{s+1}| = |A|$, $|C_{s+1}| = |C|$, $X_{s+1} = A_{s+1} \,||\, C_{s+1} \,||\, \langle |A_{s+1}| \rangle_{64} \,||\, \langle |C_{s+1}| \rangle_{64} = X_{s+1}[x] \,||\, \cdots \,||\, X_{s+1}[1]$ であり，$i = 1, \ldots, d$ に対して $X_{s+1}[2^i] = X[2^i] \oplus 2^{-s} D[i]$，且つ二冪でない j に対して $X_{s+1}[j] = X[j]$ である．加えて攻撃者にとって $T \oplus T_{s+1} = \mathtt{msb}_\tau \left(\bigoplus_{i=1}^{d} 2^{-s} D[i] L^{2^i} \right)$ は既知である．攻撃者は $A_D L$ の $\tau + s + 1$ 番目のビットを復元するために，(N, A_{s+2}, C_{s+2}) *s.t.* $|A_{s+2}| = |A|$, $|C_{s+2}| = |C|$, $X_{s+2} = A_{s+2} \,||\, C_{s+2} \,||\, \langle |A_{s+2}| \rangle_{64} \,||\, \langle |C_{s+2}| \rangle_{64} = X_{s+2}[x] \,||\, \cdots \,||\, X_{s+2}[1]$ とし，$i = 1, \ldots, d$ に対して $X_{s+2}[2^i] = X[2^i] \oplus 2^{-s-1} D[i]$，且つ j が二冪でない場合 $X_{s+2}[j] = X[j]$ とする．このとき (N, A_{s+2}, C_{s+2}) に対する有効なタグ T_{s+2} について以下が成立する．まず $\mathtt{msb}_1(T \oplus T_{s+1}) = 0$ のとき

$$T \oplus T_{s+2} = (\mathtt{lsb}_{\tau-1}(T \oplus T_{s+1}) \,||\, b)$$

が成立し，$\mathtt{msb}_1(T \oplus T_{s+1}) = 1$ のとき

$$T \oplus T_{s+2} = (\mathtt{lsb}_{\tau-1}(T \oplus T_{s+1}) \,||\, b) \oplus \mathtt{msb}_\tau(1^2 0^4 10^{120} 1)$$

が成立する．ここで b は $A_D L$ の未知の $\tau + s + 1$ 番目のビットと，$A_D L$ の既知の最初の s ビットから導出される未知のビットである．1 回の復号クエリにより b が復元され，ここから $A_D L$ の $\tau + s + 1$ 番目のビットが復元される．このステップでは $128 - \tau$ 回の復号クエリが必要であり，各クエリ 2^d ブロックの入力が必要である．

第 3 ステップ: 残りの未知ビットの回復. このステップでは L の残りの未知 $d-1$ ビットを復元するために再度 Ferguson の偽造攻撃を行う．しかし攻撃者は既に L の $128 - d + 1$ ビット情報を保持しているため，Ferguson が示したように，その情報を用いて偽造成功に必要な復号クエリ数を大幅に削減することができる．

L に関する $128 - d + 1$ 個の独立な線形方程式によって，L を $L = B \cdot L' \oplus S$ と表すことができる．ここで L' は L の未知ビットを表す $d-1$ ビット値であり，B は GF(2) 上の $128 \times (d-1)$ 行列，S は 128 ビットのビット列である．B と S は第 2 ステップまでに取得した線形方程式から導出されるため攻撃者にとって既知である．ここで (N, A^*, C^*, T^*) を，これから攻撃者が作成する有効な偽造とし，$|A^*| = |A|, |C^*| = |C|$ とする．(A, C) と (A^*, C^*) の差から定義される新しい 128×128 のエラー行列を A_{D^*} とすると，次の式が得られる．

$$\begin{aligned} T \oplus T^* &= \mathtt{msb}_\tau(A_{D^*} L) = \mathtt{msb}_\tau(A_{D^*}(B \cdot L' \oplus S)) \\ &= \mathtt{msb}_\tau(A_{D^*} B L' \oplus A_{D^*} S). \end{aligned}$$

第 1 ステップにおける最初の偽造と同様に，攻撃者は $A_{D^*} B$ の上 $\tau - 1$ 行が 0 になるように A^* と C^* を選択することができる．なぜなら攻撃者が操作できる変数の数 $128d$ よりもゼロ化する要素数 $(d-1) \times (\tau - 1)$ の方が少ないからである．これより T^* の実効タグ長は 1 ビットとなり，攻撃者は 1 回の復号クエリで L' に関する線形方程式を得ることができる．その後，攻撃者は $A_{D^*} B L'$ の値全体をビッ

© 2026 Information Processing Society of Japan

ト単位で復元してもよいし，新しいエラー行列を作成して L に関する新たな線形方程式を 1 つ得てもよい．何れの場合も，このステップには高々 $d-1$ 回の復号クエリが必要であり，各クエリで 2^d ブロックの入力が必要である．

4. 計算量評価と既存攻撃との比較

本章では 3 章で述べた提案攻撃の計算量を評価し，NIST 制限下における Ferguson/MW 攻撃との比較を行う．但し本章の評価及び比較は各攻撃における最良のケースを仮定していることに留意されたい．Ferguson/MW 攻撃は，元の入力が *partial block*，つまり元の AD 又は暗号文の長さが 128 の倍数ではないことを仮定している．3 章で述べた我々の攻撃では，元の入力が *full block*，つまり元の AD 及び暗号文の長さは 128 の倍数であることを仮定する．

3 章で述べた提案攻撃に必要な復号クエリ数及びデータ計算量は以下の通りである．

$$\text{Query: } 2^{\tau-d+1} + (128 - \tau + d - 1)$$
$$\text{Data: } 2^{\tau+1} + 2^d(128 - \tau + d - 1)$$

32/64 ビットのタグの場合における NIST 制限下でのデータ計算量はそれぞれ 2^{33} 及び 2^{65} となる．なぜなら提案攻撃では最初の偽造後は実効タグ長が 1 ビットの偽造を複数回行うだけで良く，その必要クエリ数 $(128-\tau+d-1)$ が NIST 制限値を下回るためである．一方，Ferguson/MW 攻撃では，最初の偽造後に高コストな偽造，即ち必要クエリ数が NIST の制限値を超える偽造が複数回必要となる．例えば $\tau = 32$，$\log_2 \ell = d = 6$，及び $\log_2 q = 11$ の場合，Ferguson/MW 攻撃では 5 回/4 回の高コスト偽造成功が必要であり，認証鍵回復に必要な偽造のクエリ数は以下の通りである．比較のために，提案攻撃に必要なクエリ数も併せて示す．

$$\text{Ferguson: } 2^{26}, 2^{25}, 2^{23}, 2^{18}, 2^{11}, 2^2, \underbrace{1, 1, \ldots, 1}_{23}$$
$$\text{MW: } 2^{32}, 2^{24}, 2^{22}, 2^{17}, 2^9, \underbrace{1, 1, \ldots, 1}_{24}$$
$$\text{Ours: } 2^{27}, \underbrace{1, 1, \ldots, 1}_{101}$$

提案攻撃において，最初の偽造に必要なクエリ数は Ferguson の攻撃よりも 1 ビット大きい．しかしながら Ferguson/MW 攻撃と比較して 2 回目以降の偽造攻撃コストを復号クエリ 1 回にすることで，提案攻撃は NIST 制限下における総データ計算量を大幅に削減することができる．

最初の偽造に必要なクエリ回数の増分． 表 3 に示すように，Ferguson（*resp.* MW）の攻撃において最初の偽造に必要なクエリ数及びデータ計算量はそれぞれ $2^{\tau-d}$ 及び 2^{τ}（*resp.* 2^{τ} 及び 2^{τ}）であり，提案攻撃と比較して 1 ビット小さい．なぜなら，Ferguson/MW 攻撃では partial block の場合，入力ブロック数を変更せずに長さ情報ブロックに差分を入れることができ，$D[0] := \langle |A| \rangle_{64} \,||\, \langle |C| \rangle_{64} \oplus \langle |A_1| \rangle_{64} \,||\, \langle |C_1| \rangle_{64} \neq 0^{128}$ と設定できるからである．つまり Ferguson/MW 攻撃では $D[1], \ldots, D[d]$ を操作することで A_D の上 d 行を 0 にすることができる．一方提案攻撃では $D[0] = 0^{128}$ とするため，0 にできるのは $d-1$ 行のみである．提案攻撃では第 2 ステップでエラー多項式に 2^{-1} を繰り返し掛けるため，$D[0] \neq 0^{128}$ と設定すると非ゼロの $2^{-s}D[0]$ と他のブロックとの整合性を維持することが困難となるためである．

表 3: NIST 制限下での認証鍵回復攻撃において，最初の偽造に必要な復号クエリ数とデータ計算量

	query	data
Ferguson [8]	$2^{\tau-d}$	2^{τ}
MW [15]	2^{τ}	2^{τ}
Ours (full block)	$2^{\tau-d+1}$	$2^{\tau+1}$

5. 提案攻撃の応用

5.1 認証鍵回復後の攻撃

当然ながら，攻撃者が認証鍵を入手すると GCM の改ざん検知の安全性は壊滅的な打撃を受ける．攻撃者は正当なナンス，AD，暗号文，タグの対を得たのち，暗号文と AD を自由に改ざんし，それに対応する有効なタグを偽造することが可能になる．秘匿の安全性に関しては，Joux [12] 及び Ashur ら [2] で論じられているように，96 ビットナンスのみが使用されている限り安全性は維持されるが，それ以外の場合は安全でなくなる．なぜなら 96 ビットでないナンス N が入力されると，GCM の暗号化関数は $\mathsf{GHASH}_L(\varepsilon, N)$ を計算し，その値を GCTR の初期カウンター値 Ctr0 として用いるため，認証鍵 L を持つ攻撃者は Ctr0 の値を意図的に衝突させることが可能となるためである．

5.2 Partial block の場合への適用

元の AD または暗号文の長さが 128 の倍数でない場合，提案攻撃に必要なクエリ数は増加しうる．4 章で述べた通り，提案攻撃では $D[0] = 0^{128}$，つまり元の AD 及び暗号文と最初の偽造で用いる AD 及び暗号文の長さは同じである必要がある．差分を設定したい $D[\cdot]$ に対応するブロックが partial の場合，攻撃者は基本的にそのブロックに対して所望の差分を設定できないため，そのようなブロックは攻撃に使用することができず，必要クエリ数が増加する．必要クエリ数の増分は d に依存する．

$d \geq 3$ の場合，攻撃者は少なくとも $(d-2)$ 個の $D[\cdot]$ を使用できるため，それらのブロックを用いて 3 章の攻撃を実行することができる．これにより最初の偽造に必要なクエリ数は高々 2 ビット増加することとなる．$d \leq 2$ の場

© 2026 Information Processing Society of Japan

合，攻撃者が少なくとも 1 つの $D[\cdot]$ を使用できるならば，$d \geq 3$ の場合と同様に攻撃を実行できる．そうでない場合，攻撃者はどの $D[\cdot]$ も使用できず，3 章の攻撃を直接適用することはできない．この場合，我々の攻撃は以下のように修正することができる．まず第 1 ステップで攻撃者は 2 回の Ferguson 型偽造攻撃を実行する．つまり $D[0]$ に差分を入れて，最初の偽造と 2 番目の偽造の入力が full block であるようにする．但し，この 2 回の偽造攻撃で攻撃者は AD 長と暗号文長を同一に設定する．その結果，攻撃者は，入力が full block であり，かつ $|A_1| = |A_2|$, $|C_1| = |C_2|$ を満たす相異なる正当な偽造 (N, A_1, C_1, T_1), (N, A_2, C_2, T_2) を取得する．もしも通常の Ferguson 攻撃を適用することでこの 2 つの偽造が同一となってしまう場合は，2 つ目の偽造では error matrix の上 d 行をゼロ化するのではなく，異なる箇所の d 行をゼロ化すればよい．その後，$T_1 \oplus T_2$ に 2^{-1} を掛けることで，3 章で述べた攻撃の第 2 ステップに戻ることができる．上記攻撃では最初の偽造だけでなく 2 番目の偽造も高コストとなる．これにより表 1 に示すように，NIST 制限下において $d \geq 3$ の場合と比較して，必要データ計算量がやや増加する．なお，必要データ計算量の導出方法は MW [15] に倣った．

5.3 既存認証鍵回復攻撃への応用

GHASH のような多項式ハッシュを用いた認証方式に対する認証鍵絞り込み/回復攻撃は広く研究されてきた [1, 9, 13, 14, 16, 19, 20, 22] が，これらの既存解析は全て短縮タグの場合を考慮していない．なぜならタグ短縮により多項式の定数項に未知ビットが発生してしまい，上記の解析に必要な数学的構造に自由度が生まれて解析の効果が下がるためである．提案攻撃は上記既存解析の中で，復号クエリを用いて認証鍵絞り込み/回復攻撃を行う攻撃 [1, 9, 19, 20, 22] と組み合わせることができる．詳細は割愛するが，既存解析における偽造攻撃を 3 章で述べた提案攻撃の第 1 ステップとして用いれば，その後第 2 ステップで $128 - \tau$ 回の復号クエリを実行することでエラー多項式が復元でき，既存解析における短縮されていないタグの場合の解析に接続することができる．

5.4 wGCM への適用

wGCM [17] は現在 NIST 標準文書改訂において仕様が議論されている方式で，GCM の内部ブロック暗号の入出力長を 128 ビットから 256 ビットに拡張した方式である．現在 wGCM の認証部分の設計として議論に上がっているのは，GHASH を 256 ビット有限体に自然に拡張する wide-GHASH と，現在の 128 ビット GHASH を独立な鍵を 2 つ用いて 2 回実行し，それらを連結させる concat-GHASH の 2 つである．厳密な仕様は議論中であるものの，NIST はタグ短縮の可能性に言及しており，その場合提案攻撃はどちらにも基本的に適用可能であると考えられる．

6. ナンス誤用設定における認証鍵回復攻撃

本章では 3 章の提案攻撃をナンス誤用設定下の場合に適用する．これは Joux の forbidden attack [12] を初めて短縮タグの場合に拡張したものである．まず比較のため Joux の攻撃の自然な拡張を示し，その後提案攻撃の拡張を示す．

6.1 Joux 攻撃の自然な拡張

Joux の攻撃 [12] では，同じナンスを用いた 2 つの正当なタプル (N, A_1, C_1, T_1), (N, A_2, C_2, T_2) を入手した受動的攻撃者が方程式 $\mathsf{GHASH}_L(A_1, C_1) \oplus \mathsf{GHASH}_L(A_2, C_2) = T_1 \oplus T_2$ を得られることを利用する．この方程式の次数が ℓ のとき，方程式を解くことで高々 ℓ 個の認証鍵候補が得られるので，追加の暗号化/復号クエリを用いて本物の認証鍵を絞り込む．本節では τ ビットタグの場合にこの攻撃を自然に拡張する．このとき攻撃者が得ているのは次の方程式である．

$$\mathsf{GHASH}_L(A_1, C_1) \oplus \mathsf{GHASH}_L(A_2, C_2) = (T_1 \oplus T_2) \,||\, Y.$$

ここで Y は攻撃者にとって未知の $128 - \tau$ ビット値である．攻撃者は Y の取りうる値を全て試すことで Joux の攻撃を実行することができる．必要計算量は表 2 の通りであり，ℓ 次の方程式を $2^{128-\tau}$ 回解くことで高々 $2^{128-\tau}\ell$ 個の認証鍵候補が得られ，追加のクエリを用いて本物の認証鍵を絞り込む．ここで，認証鍵の絞り込みに必要なクエリ数（表 2 中の $\mathsf{find}(\ell)$）は攻撃者が実行可能なクエリによって変わる．復号クエリを用いる場合，攻撃者は鍵候補を根としてエラー多項式を生成し，そのエラー多項式を用いて偽造攻撃を行うことで本物の認証鍵を絞り込む．GCM の入力長制限を超えない限りは二分探索で認証鍵を絞り込むことができるため，$\mathsf{find}(\ell)$ の下界は $\log_2(\ell)$ となる．但しこれは短縮されていないタグの場合の評価であり，短縮タグの場合は必要クエリ数は更に増加すると考えられる．ナンス誤用暗号化クエリを用いる場合，攻撃者は鍵候補毎に $E_K(\mathsf{Ctr0})$ を導出し，その値を用いて新たに得たタグとの整合性をチェックする．1 回の暗号化クエリで平均的に鍵候補数を $1/2^{\tau}$ 倍にできるため，必要クエリ数はおおよそ $\log_2(\ell)/\tau$ となる．

6.2 提案攻撃のナンス誤用設定への拡張

3 章で述べた提案攻撃の第 1 ステップ，即ち $2^{\tau-d+1}$ 回の復号クエリを用いて最初の偽造攻撃を行う作業は，1 回のナンス誤用暗号化クエリに置き換えることができる．よって第 2 ステップで示したように $128 - \tau$ 回の復号クエリでエラー多項式が回復でき，その結果 Joux の攻撃を適用することが可能となる．また，第 2 ステップにおけるエラー多項式回復のためには AD 長と暗号文長がオリジナルと同一且つ full block であるという条件のみが必要であ

© 2026 Information Processing Society of Japan

り，他の要件を必要としない．つまり，攻撃者は任意の正当な full block タプル (N, A_1, C_1, T_1), (N, A_2, C_2, T_2) *s.t.* $|A_1| = |A_2|$, $|C_1| = |C_2|$ が得られさえすれば，Joux の攻撃と同じ攻撃者設定で攻撃を実行できる．この提案攻撃の拡張に必要な計算量は表 2 の通りである．Joux の攻撃と比較すると，提案攻撃の拡張は追加の復号クエリを $128 - \tau$ 回実行するだけで適用タグ長を任意の τ とできることが分かる．節 6.1 で述べた Joux の自然な拡張は方程式を解く回数が格段に多く，且つ鍵候補の絞り込みについて，復号クエリを用いる場合の $\mathsf{find}(\ell)$ の下界が $\log_2(\ell)$ であることを考慮すると，提案攻撃の拡張よりも追加クエリ数が少なくなることはない．

7. おわりに

本稿では NIST 標準認証暗号 GCM に対する認証鍵回復攻撃を提案した．提案攻撃は τ ビットタグ GCM に対して，NIST の何れの制限下でもデータ計算量 $O(2^\tau + 1)$ を達成する．つまり提案攻撃は制限のない場合の必要データ計算量 $O(2^\tau)$ とほぼ同じ計算量を達成しており，NIST の制限が認証鍵回復の計算量増加にほぼ寄与していないことを示している．また提案攻撃をナンス誤用設定に拡張することで，Joux の forbidden attack を初めて短縮タグの場合に拡張した．

参考文献

[1] M. A. Abdelraheem, P. Beelen, A. Bogdanov, and E. Tischhauser. Twisted polynomials and forgery attacks on GCM. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 762–786. Springer, Berlin, Heidelberg, Apr. 2015.

[2] T. Ashur, O. Dunkelman, and A. Luykx. Boosting authenticated encryption robustness with minimal modifications. In J. Katz and H. Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 3–33. Springer, Cham, Aug. 2017.

[3] M. Bellare and C. Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In T. Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 531–545. Springer, Berlin, Heidelberg, Dec. 2000.

[4] D. Black and D. McGrew. Using Authenticated Encryption Algorithms with the Encrypted Payload of the Internet Key Exchange version 2 (IKEv2) Protocol. RFC 5282, 2008. Accessed: 2026-07-23.

[5] M. Campagna, A. Maximov, and J. P. Mattsson. Galois Counter Mode with Strong Secure Tags (GCM-SST). Internet-Draft draft-mattsson-cfrg-aes-gcm-sst-21, Internet Engineering Task Force, July 2026. Work in Progress.

[6] J. Daemen and V. Rijmen. *The Design of Rijndael: The Advanced Encryption Standard (AES)*. Information Security and Cryptography. Springer Berlin, Heidelberg, 2 edition, 2020.

[7] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST-SP 800-38D, 2007.

[8] N. Ferguson. Authentication Weaknesses in GCM. Public Comments to NIST, 2005. http://csrc.nist.gov/groups/ST/toolkit/BCM/comments.html.

[9] H. Handschuh and B. Preneel. Key-recovery attacks on universal hash function based MAC algorithms. In D. Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 144–161. Springer, Berlin, Heidelberg, Aug. 2008.

[10] A. Inoue, A. Jha, B. Mennink, and K. Minematsu. Generic security of GCM-SST. In M. Fischlin and V. Moonsamy, editors, *ACNS 2025, Part II*, volume 15826 of *LNCS*, pages 342–369. Springer, Cham, June 2025.

[11] T. Iwata, K. Ohashi, and K. Minematsu. Breaking and repairing GCM security proofs. In R. Safavi-Naini and R. Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 31–49. Springer, Berlin, Heidelberg, Aug. 2012.

[12] A. Joux. Authentication Failures in NIST version of GCM. Public Comments to NIST, 2006. http://csrc.nist.gov/groups/ST/toolkit/BCM/comments.html.

[13] G. Leurent and F. Sibleyras. The missing difference problem, and its applications to counter mode encryption. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 745–770. Springer, Cham, Apr. / May 2018.

[14] A. Luykx and B. Preneel. Optimal forgeries against polynomial-based MACs and GCM. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 445–467. Springer, Cham, Apr. / May 2018.

[15] J. Mattsson and M. Westerlund. Authentication key recovery on Galois/counter mode (GCM). In D. Pointcheval, A. Nitaj, and T. Rachidi, editors, *AFRICACRYPT 16*, volume 9646 of *LNCS*, pages 127–143. Springer, Cham, Apr. 2016.

[16] M. Nandi. Bernstein bound on WCS is tight - repairing luykx-preneel optimal forgeries. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 213–238. Springer, Cham, Aug. 2018.

[17] National Institute of Standards and Technology. NIST to Revise Special Publication 800-38D, 2026. https://csrc.nist.gov/pubs/sp/800/38/d/r1/2prd.

[18] Y. Niwa, K. Ohashi, K. Minematsu, and T. Iwata. GCM security bounds reconsidered. In G. Leander, editor, *FSE 2015*, volume 9054 of *LNCS*, pages 385–407. Springer, Berlin, Heidelberg, Mar. 2015.

[19] G. Procter and C. Cid. On weak keys and forgery attacks against polynomial-based MAC schemes. In S. Moriai, editor, *FSE 2013*, volume 8424 of *LNCS*, pages 287–304. Springer, Berlin, Heidelberg, Mar. 2014.

[20] G. Procter and C. Cid. On weak keys and forgery attacks against polynomial-based MAC schemes. *Journal of Cryptology*, 28(4):769–795, Oct. 2015.

[21] P. Rogaway. Authenticated-encryption with associated-data. In V. Atluri, editor, *ACM CCS 2002*, pages 98–107. ACM Press, Nov. 2002.

[22] M.-J. O. Saarinen. Cycling attacks on GCM, GHASH and other polynomial MACs and hashes. In A. Canteaut, editor, *FSE 2012*, volume 7549 of *LNCS*, pages 216–225. Springer, Berlin, Heidelberg, Mar. 2012.

[23] J. Viega and D. McGrew. The Use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP). RFC 4106, 2005. Accessed: 2026-07-23.

© 2026 Information Processing Society of Japan

付　　録

A.1　GCM の暗号化図

GCM の暗号化を図 A·1 に示す.

A.2　ナンス誤用選択平文攻撃下における認証鍵回復攻撃

6.1, 6.2 節で示した Joux の攻撃とその自然な拡張，及び提案攻撃の拡張では，攻撃者が同じナンスを用いた 2 つの正当なタプル (N, A_1, C_1, T_1), (N, A_2, C_2, T_2) を得るという設定を用いていた．本節ではそれよりも強い攻撃者に対する短縮タグの場合の認証鍵回復攻撃を 2 つ示す.

まず攻撃者が選択平文攻撃を実行できる場合，3 章の提案攻撃を以下のように簡易化できる．この攻撃は GCM-SST に対する認証サブ鍵回復攻撃とほぼ同様の手順である.

(1) M を任意の 128 ビットの平文とする．任意のナンス N 及び AD A に対して, 2 つの暗号化クエリ (N, A, M) 及び $(N, A, M \oplus 1)$ を実行し，(C_1, T_1), (C_2, T_2), 及び $T_1 \oplus T_2 = \mathtt{msb}_\tau(L^2)$ を取得する.

(2) 復号クエリによって L^2 をビット毎に回復する．$C_3 = C_1 \oplus 2^{-1}$ とすると，(N, A, C_3) に対する正当なタグ T_3 は $T_1 \oplus T_3 = \mathtt{msb}_\tau(2^{-1}L^2) = (\mathtt{lsb}_{\tau-1}(T_1 \oplus T_2) \,||\, b) \oplus (\mathtt{msb}_\tau(1^2 0^4 1 0^{120} 1) \cdot \mathtt{msb}_1(T_1 \oplus T_2))$ を満たす．ここで，b は L^2 における未知の $\tau+1$ 番目のビットである．よって 1 回の復号クエリで b が復元できる.

(3) 上記の手順を繰り返して L^2 の値全てを復元する．L^2 の上位 $\tau + s$ ビットが復元されていると仮定する．これより，$C_{s+2} = C_1 \oplus 2^{-s}$ 及びこれに対応する正当なタグ T_{s+2} について，$T_1 \oplus T_{s+2} = \mathtt{msb}_\tau(2^{-s}L^2)$ の値は既知である．$C_{s+3} = C_1 \oplus 2^{-s-1}$ と設定すると，対応する正当なタグ T_{s+3} は $T_1 \oplus T_{s+3} = \mathtt{msb}_\tau(2^{-s-1}L^2) = (\mathtt{lsb}_{\tau-1}(T_1 \oplus T_{s+2}) \,||\, b) \oplus (\mathtt{msb}_\tau(1^2 0^4 1 0^{120} 1) \cdot \mathtt{msb}_1(T_1 \oplus T_{s+2}))$ を満たす．ここで b は L^2 の $(\tau + s + 1)$ 番目のビットから導出される未知のビットである.

$L \mapsto L^2$ は $\mathrm{GF}(2^{128})$ における置換であるため，L^2 から L を復元することができる．上記攻撃に必要な計算量は，ナンス誤用でない暗号化クエリ 1 回，ナンス誤用暗号化クエリ 1 回，及び復号クエリ $128 - \tau$ 回である．なお，上記の攻撃における最初のナンス誤用でない暗号化クエリは複数ブロックの場合に容易に拡張でき，且つ既知平文でよいことに注意されたい.

次に攻撃者が選択平文のナンス誤用暗号化クエリを**複数回**行える場合, 攻撃は更に簡単になり, 復号クエリも不要となる．まず攻撃者は新しいナンスを用いて 1 回の暗号化クエリを実行し, その後 $\lceil 128/\tau \rceil$ 回のナンス誤用暗号化クエリを実行することで L を復元することができる．例えば $\tau = 32$ のとき, 攻撃者は (N, A, M), $(N, A, M \oplus 1)$, $(N, A, M \oplus 2^{-32})$, $(N, A, M \oplus 2^{-64})$, $(N, A, M \oplus 2^{-96})$ を暗号化オラクルに問い合わせ，(C_1, T_1), (C_2, T_2), …, (C_5, T_5) を得る．これより $i = 2, \ldots, 5$ に対して $T_1 \oplus T_i = \mathtt{msb}_{32}(2^{-32(i-2)}L^2)$ となり，L^2 の値が 32 ビットずつ明らかになるため，攻撃者は L^2 の値全体を復元することができる.

© 2026 Information Processing Society of Japan

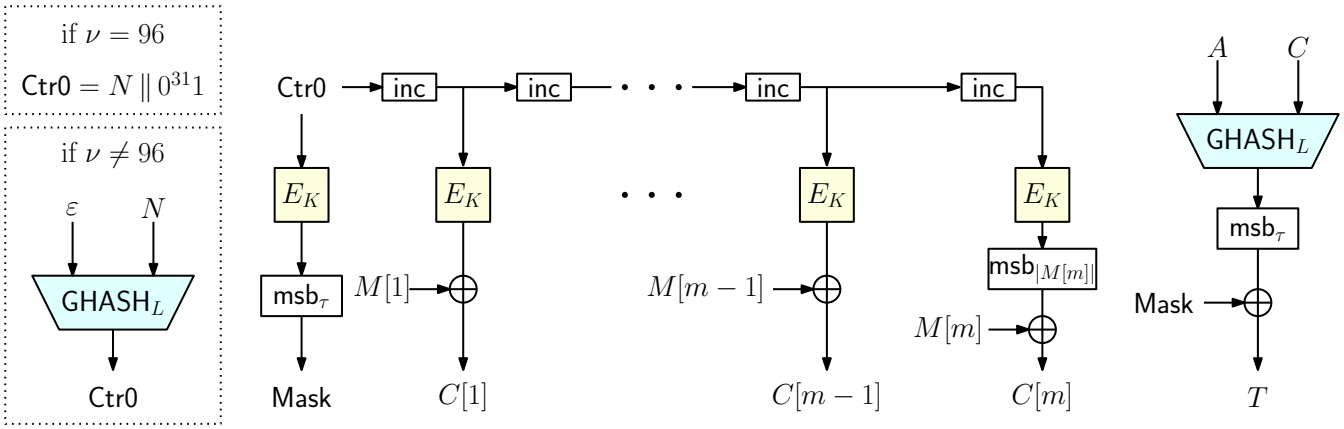


図 A·1: GCM の暗号化.

© 2026 Information Processing Society of Japan